

Practical 8.1

Aim: **Sliding Window Protocol** (Go-Back-N variant)

This example includes:

Sender side

Receiver side (simulated)

Uses arrays and simple logic to simulate sending, acknowledgment, and sliding of window.

Algorithm: Sliding Window Protocol (Go-Back-N ARQ)

Input:

- Total number of frames to send N
- Window size w

Steps:

1. **Initialize:**
 - Set $base = 0$ (first frame in window)
 - Set $next_seq_num = 0$ (next frame to send)
2. **While $base < N$:**
 - While $next_seq_num < base + w$ and $next_seq_num < N$:
 - Send frame with sequence number $next_seq_num$
 - Increment $next_seq_num = next_seq_num + 1$
3. **Wait for acknowledgment:**
 - If ACK is received for frame $base$:
 - Increment $base = base + 1$
 - Else (timeout or error detected):
 - Retransmit all frames from $base$ up to $next_seq_num - 1$
4. **Repeat** until all frames are sent and acknowledged ($base \geq N$)
5. End

Explanation:

- The sender can send up to w unacknowledged frames.
- If an error or timeout occurs for a frame, the sender **goes back** and retransmits from that frame onward.
- Receiver keeps track of the expected sequence number and discards out-of-order frames.

Additional (Receiver side):

1. Initialize `expected_seq_num = 0`
2. When a frame with sequence number `seq_num` arrives:
 - o If `seq_num == expected_seq_num`:
 - Accept the frame
 - Deliver it to higher layer
 - Send ACK for `expected_seq_num`
 - Increment `expected_seq_num`
 - o Else:
 - Discard frame
 - Resend ACK for `expected_seq_num - 1`

Summary:

- Go-Back-N allows the sender to transmit multiple frames before waiting for ACKs.
- Upon error, retransmission starts from the frame where the error occurred.
- Efficient for networks with fewer errors and longer delays.

C Program: Sliding Window Protocol (Go-Back-N)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_FRAMES 50

void sender(int frames[], int size, int window_size);
void receiver(int frames[], int start, int window_size);

int main() {
    int n, window_size;
    int frames[MAX_FRAMES];

    printf("Sliding Window Protocol Simulation (Go-Back-N)\n");
    printf("Enter number of frames to send (max %d): ", MAX_FRAMES);
    scanf("%d", &n);

    if(n > MAX_FRAMES || n <= 0) {
        printf("Invalid number of frames.\n");
        return 0;
    }

    printf("Enter window size: ");
    scanf("%d", &window_size);
```

```

    if(window_size <= 0 || window_size > n) {
        printf("Invalid window size.\n");
        return 0;
    }

    // Fill frames with sequence numbers
    for(int i=0; i<n; i++) {
        frames[i] = i;
    }

    sender(frames, n, window_size);

    return 0;
}

void sender(int frames[], int size, int window_size) {
    int base = 0;
    int next_seq = 0;
    char choice;

    srand(time(NULL));

    while(base < size) {
        printf("\nSender window: ");
        for(int i=base; i<base+window_size && i<size; i++) {
            printf("[%d] ", frames[i]);
        }
        printf("\nSending frames...\n");

        for(int i=base; i<base+window_size && i<size; i++) {
            printf("Sent frame: %d\n", frames[i]);
        }

        printf("\nSimulating receiver...\n");
        receiver(frames, base, window_size);

        printf("\nDid all frames receive correctly? (y/n): ");
        scanf(" %c", &choice);

        if(choice == 'y' || choice == 'Y') {
            base += window_size;
        } else {
            printf("Resending from frame %d...\n", frames[base]);
            // In Go-Back-N, we resend from base
        }
    }

    printf("\nAll frames sent successfully!\n");
}

void receiver(int frames[], int start, int window_size) {
    for(int i=start; i<start+window_size; i++) {
        printf("Received frame: %d, sending ACK: %d\n", frames[i],
frames[i]+1);
    }
}

```

How it works:

- The sender keeps track of a `base` and a `window_size`.
- It sends frames within `[base, base+window_size)`.
- After sending, it asks the user if all frames were received correctly.
- If yes, it slides the window forward by `window_size`.
- If not, it resends starting from `base`.

How to run:

1. Save as `sliding_window.c`.
2. Compile:

```
gcc sliding_window.c -o sliding_window
```

3. Run:

```
./sliding_window
```

Open ▾ ↵

sliding_window.c
/home/sit-lab1

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #define MAX_FRAMES 50
5 void sender(int frames[], int size, int window_size);
6 void receiver(int frames[], int start, int window_size);
7 int main() {
8     int n, window_size;
9     int frames[MAX_FRAMES];
10    printf("Sliding Window Protocol Simulation (Go-Back-N)\n");
11    printf("Enter number of frames to send (max %d): ", MAX_FRAMES);
12    scanf("%d", &n);
13    if(n > MAX_FRAMES || n <= 0) {
14        printf("Invalid number of frames.\n");
15        return 0;
16    }
17    printf("Enter window size: ");
18    scanf("%d", &window_size);
19    if(window_size <= 0 || window_size > n) {
20        printf("Invalid window size.\n");
21        return 0;
22    }
23    // Fill frames with sequence number
24    for(int i=0; i<n; i++) {
25        frames[i] = i;
26    }
27    sender(frames, n, window_size);
28    return 0;
29 }
30 void sender(int frames[], int size, int window_size) {
31     int base = 0;
32     int next_seq = 0;
33     char choice;
34     srand(time(NULL));
35     while(base < size) {
36         printf("\nSender window: ");
37         for(int i=base; i<base+window_size && i<size; i++) {
38             printf("[%d] ", frames[i]);
39         }
40         printf("\nSending frames...\n");
41         for(int i=base; i<base+window_size && i<size; i++) {
42             printf("Sent frame: %d\n", frames[i]);
43         }
44         printf("\nSimulating receiver...\n");
```

```

18     scanf("%d", &window_size);
19     if(window_size <= 0 || window_size > n) {
20         printf("Invalid window size.\n");
21         return 0;
22     }
23     // Fill frames with sequence number
24     for(int i=0; i<n; i++) {
25         frames[i] = i;
26     }
27     sender(frames, n, window_size);
28     return 0;
29 }
30 void sender(int frames[], int size, int window_size) {
31     int base = 0;
32     int next_seq = 0;
33     char choice;
34     srand(time(NULL));
35     while(base < size) {
36         printf("\nSender window: ");
37         for(int i=base; i<base+window_size && i<size; i++) {
38             printf("[%d] ", frames[i]);
39         }
40         printf("\nSending frames...\n");
41         for(int i=base; i<base+window_size && i<size; i++) {
42             printf("Sent frame: %d\n", frames[i]);
43         }
44         printf("\nSimulating receiver...\n");
45         receiver(frames, base, window_size);
46         printf("\nDid all frames receive correctly? (y/n): ");
47         scanf(" %c", &choice);
48         if(choice == 'y' || choice == 'Y') {
49             base += window_size;
50         } else {
51             printf("Resending from frame %d...\n", frames[base]);
52             // In Go-Back-N, we resend from base
53         }
54     }
55     printf("\nAll frames sent successfully!\n");
56 }
57 void receiver(int frames[], int start, int window_size) {
58     for(int i=start; i<start+window_size; i++) {
59         printf("Received frame: %d, sending ACK: %d\n", frames[i], frames[i]+1);
60     }
61 }

```

Output:

```
sit-lab1@sit-lab1-OptiPlex-3280-AIO:~$ gedit sliding_window.c
Gtk-Message: 10:06:48.855: Not loading module "atk-bridge": The functionality is provided by GTK natively. Please try to not load it.
(gedit:4899): GLib-GIO-WARNING **: 10:06:49.353: Error creating IO channel for /proc/self/mountinfo: Permission denied (g-file-error-quark, 2)
sit-lab1@sit-lab1-OptiPlex-3280-AIO:~$ gcc sliding_window.c -o sliding_window
sit-lab1@sit-lab1-OptiPlex-3280-AIO:~$ ./sliding_window
Sliding Window Protocol Simulation (Go-Back-N)
Enter number of frames to send (max 50): 25
Enter window size: 4

Sender window: [0] [1] [2] [3]
Sending frames...
Sent frame: 0
Sent frame: 1
Sent frame: 2
Sent frame: 3

Simulating receiver...
Received frame: 0, sending ACK: 1
Received frame: 1, sending ACK: 2
Received frame: 2, sending ACK: 3
Received frame: 3, sending ACK: 4

Did all frames receive correctly? (y/n): y

Sender window: [4] [5] [6] [7]
Sending frames...
Sent frame: 4
Sent frame: 5
Sent frame: 6
Sent frame: 7

Simulating receiver...
Received frame: 4, sending ACK: 5
Received frame: 5, sending ACK: 6
Received frame: 6, sending ACK: 7
Received frame: 7, sending ACK: 8
```